

ГУАП

КАФЕДРА № 44

ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

А.В. Аксенов

инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОЙ РАБОТЕ

«Разработка веб-приложения»

по курсу: БАЗЫ ДАННЫХ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4241

подпись, дата

А. Д. Антрушина

инициалы, фамилия

Санкт-Петербург 2024

ГУАП

КАФЕДРА № 44

ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

А.В. Аксенов

инициалы, фамилия

ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

«Разработка веб-приложения»

по курсу: БАЗЫ ДАННЫХ

ЗАДАНИЕ ПРИНЯЛ К ИСПОЛНЕНИЮ

СТУДЕНТ гр. №

4241

подпись, дата

А. Д. Антрушина

инициалы, фамилия

Санкт-Петербург 2024

1.1 Тема курсовой работы

Веб-приложение «ВУЗ: учебный план и расписание».

1.2 Словесное описание предметной области и актуальность

Учебный план и расписание являются основными элементами образовательного процесса в ВУЗе. При этом нередко возникают конфликтные ситуации, связанные, например, с ограниченным количеством аудиторий и преподавателей, несогласованностью между изучаемыми дисциплинами. Разрабатываемое веб-приложение позволит автоматизировать управление расписанием и учебным планом, обеспечить их доступность для студентов.

1.3 Описание данных, хранящихся в базе данных

База данных содержит следующие данные:

- список направлений, включая код и наименование;
- учебный план для каждого направления, включая предмет, количество часов, тип аттестации и год утверждения плана;
- список групп, включая номер, направление и номер текущего семестра;
- расписание занятий на текущий семестр, включая тип и день недели, номер пары и ФИО преподавателя;
- список преподавателей, включая ФИО, должность и номер кафедры.

1.4 Роли пользователей приложения

- пользователь без авторизации;
- администратор.

1.5 Развернутое описание функционала приложения для каждой из ролей

Система содержит информацию о направлениях, группах, преподавателях и учебном плане. Эти данные доступны для редактирования только пользователю «Администратор».

- Пользователь без авторизации

Пользователь без авторизации может смотреть расписание групп, преподавателей и аудиторий, направления и их учебные планы и список преподавателей с указанием их кафедры и должности.

- Администратор

После авторизации в системе администратор может просматривать список групп, а также изменять учебный план, расписание и информацию о преподавателях и группах.

1.6 Диаграмма вариантов использования

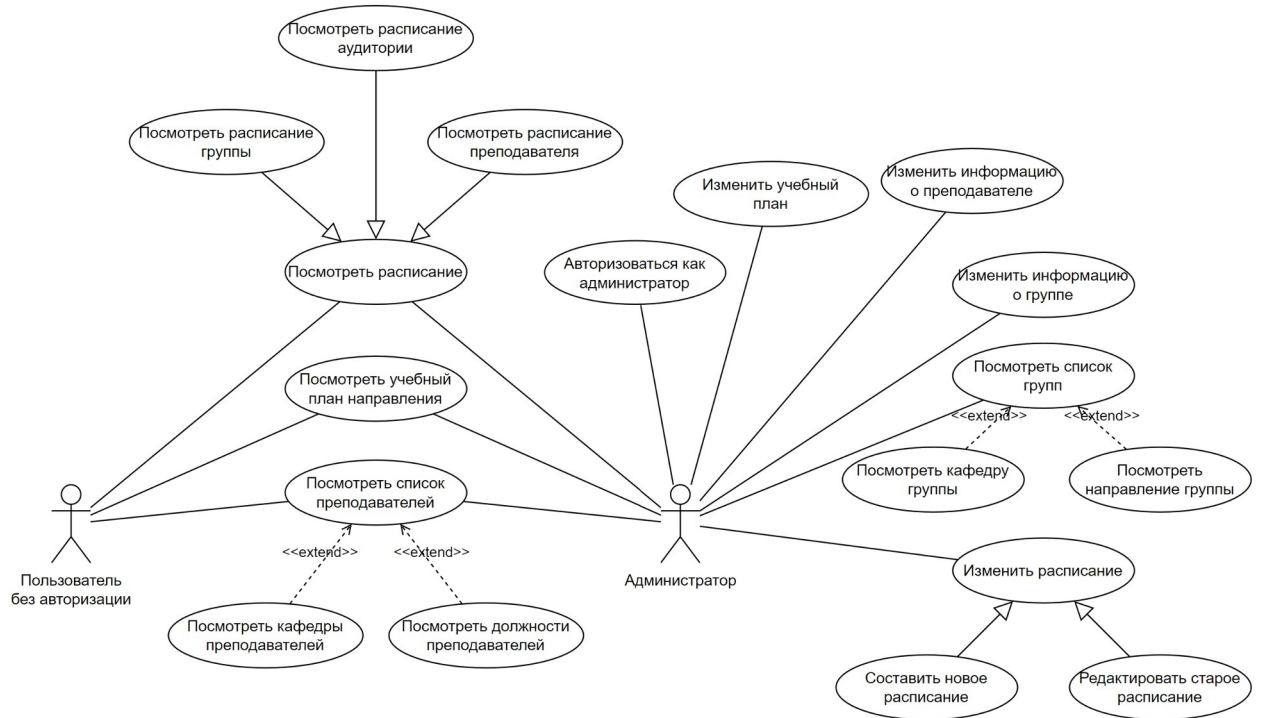


Рисунок 1 — Диаграмма вариантов использования

1.7 Предполагаемые технологии и платформа реализации

- СУБД: PostgreSQL;
- ОС: Windows 11;
- язык программирования: Python;
- фреймворк: Flask;
- тип приложения: веб-приложение.

1.8 Срок представления курсовой работы

27.12.2024

2 Схема «сущность-связь» разработанной базы данных

В предметной области «ВУЗ: учебный план и расписание» были выделены следующие сущности: Направление, Поток направления, Группа, Кафедра, Преподаватель, Предмет, Пара, Корпус, Аудитория, Семестр у потока по предмету, а также не имеющие связи с остальными сущностями Администратор и День (используется для сортировки дней недели по порядку).

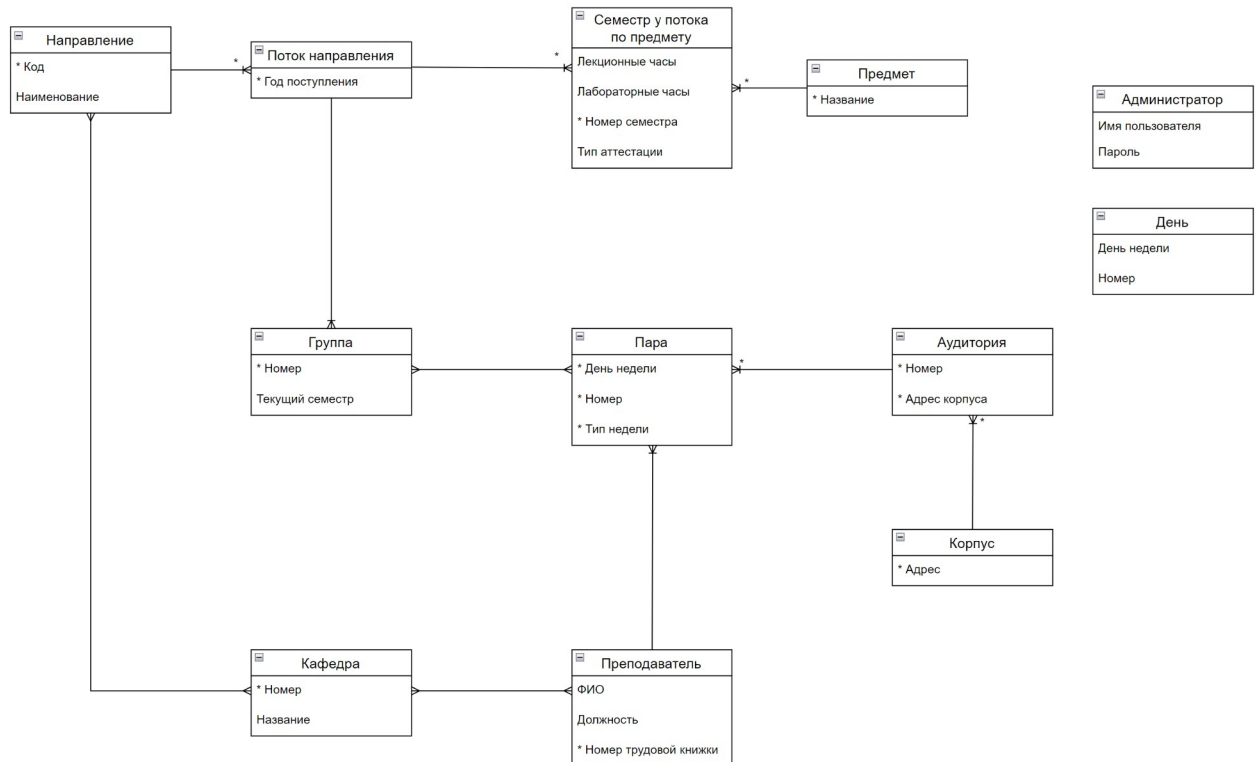


Рисунок 2 — ER-диаграмма базы данных

3 Реляционная схема разработанной базы данных

Перед созданием базы данных сначала необходимо перейти от ER-модели к реляционной модели.

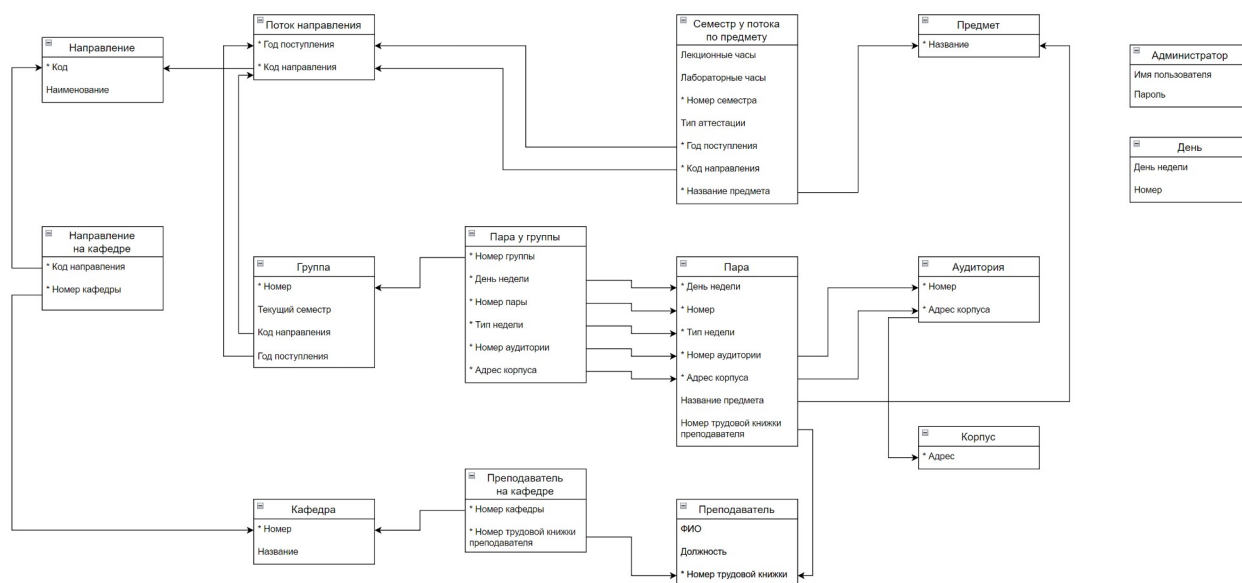


Рисунок 3 — Реляционная схема базы данных

4 Описание выбранных технологий реализации

В качестве СУБД выбрана PostgreSQL 17. Данная СУБД является реляционной и функционирует в архитектуре клиент-сервер. В качестве клиента использовано приложение DBeaver.

Для реализации веб-приложения на Python выбран фреймворк Flask. Он предоставляет минимальный набор инструментов, однако есть возможность подключать большое количество библиотек. Flask также предоставляет встроенный веб-сервер, который можно использовать на этапе разработки.

5 Описание используемых методов взаимодействия с базой данных

Для работы с базой данных использована библиотека `psycopg2`.

Для подключения к базе данных и выполнения запросов используется экземпляр объекта `Connection`. Объект `Connection` создается функцией `connect`, которая принимает в качестве аргумента параметры, используемые для подключения: имя хоста, номер порта, имя пользователя, пароль, имя базы данных. В конце работы с базой данных подключение необходимо закрыть с помощью метода `close`. Однако если объект `Connection` используется как менеджер контекста, то есть в блоке `with`, то фиксация транзакций и закрытие подключения осуществляются автоматически.

С помощью метода объекта подключения `cursor` можно создавать объекты класса `Cursor`, которые позволяют делать запросы к базе данных и обрабатывать результаты. После этого для выполнения запроса необходимо использовать метод `execute`, принимающий в качестве аргумента запрос и его параметры. Если запрос возвращает данные, то по ним можно итерировать с помощью методов `fetchone` (возвращает одну запись), `fetchmany` (возвращает указанное количество записей) и `fetchall` (возвращает все записи).

В случае, если нужно параметризовать запрос, то во избежание SQL-инъекций необходимо передавать значения параметров запроса вторым аргументом функции `execute`, при этом сам запрос и его параметры передаются на сервер отдельно, и их связывание происходит на сервере перед выполнением.

6 Описание методов развертывания приложения

При развертывании базы данных выбрана модель DBaaS (база данных как услуга). Использован бесплатный сервис Tembo, для работы с которым сначала необходимо зарегистрироваться. Затем можно приступить к созданию экземпляра базы данных: выбрать стек, версию PostgreSQL, облачного провайдера, регион (в данном случае был выбран регион Europe). После завершения процесса развертывания можно получить данные для подключения.

При развертывании приложения выбрана модель PaaS (платформа как услуга). Использован бесплатный сервис Render. После регистрации можно развернуть сервис, при этом есть возможность в качестве исходного кода выбрать git-репозиторий. Также необходимо указать команду для сборки и запуска проекта и создать файл с переменными окружения. После этого сервис автоматически осуществляет сборку и запуск проекта и он становится доступен по указанной ссылке.

7 Листинги модулей приложения

7.1 Основной модуль приложения

main.py

```
from app import app
```

__init__.py

```
from flask import Flask
```

```
from flask_bootstrap import Bootstrap5
```

```
from app.config import Config
```

```
from flask_login import LoginManager
```

```
# инициализация приложения
```

```
app = Flask(__name__)
```

```
# подключение стилей
```

```
bootstrap = Bootstrap5(app)
```

```
# создание объекта класса LoginManager
```

```
login_manager = LoginManager()
```

```
login_manager.init_app(app)
```

```
# загрузка конфигурации
```

```
app.config.from_object(Config)
```

```
# импортирование представлений
```

```
from app import routes
```

```
from app import groups
```

```
from app import teachers
```

```
from app import directions
```

```
from app import edPlanShow
```

```
from app import courses
```

```
from app import scheduleForm
```

```
from app import scheduleShow
```

```
from app import editGroup
```

```
from app import updateGroupForm
```

```
from app import insertGroupForm
```

```
from app import deleteGroupForm
```

```

from app import editStream
from app import insertStreamForm
from app import deleteStreamForm
from app import editTeacher
from app import updateTeacherForm
from app import insertTeacherForm
from app import deleteTeacherForm
from app import editSchedule
from app import insertScheduleForm
from app import deleteScheduleForm
from app import updateSchedule
from app import updateScheduleForm
from app import editEdPlan
from app import insertPlanForm
from app import deletePlanForm
from app import deleteEdplan
from app import updateEdplan
from app import updatePlanForm
from app import editCourse
from app import updateCourseForm
from app import insertCourseForm
from app import deleteCourseForm
from app import adminRegForm
from app import adminLoginForm
from app import adminLogout

```

config.py

```

import os
from dotenv import load_dotenv

load_dotenv()

class Config(object):
    DB_SERVER = os.environ.get('DB_SERVER')
    DB_NAME = os.environ.get('DB_NAME')
    DB_USER = os.environ.get('DB_USER')
    DB_PASSWORD = os.environ.get('DB_PASSWORD')
    SECRET_KEY = os.environ.get('SECRET_KEY')
    DB_SCHEMA = os.environ.get('DB_SCHEMA')
    DB_PORT = os.environ.get('DB_PORT')

```

routes.py

```
from app import app
from flask import render_template
from flask_login import current_user

@app.route("/")
def index():
    if current_user.is_authenticated:
        return render_template('index_admin.html', title='Добро пожаловать!')
    else:
        return render_template('index.html', title='Добро пожаловать!')
```

forms.py

```
from flask_wtf import FlaskForm
from wtforms import BooleanField, StringField, IntegerField, PasswordField, SubmitField, SelectField, validators
```

форма регистрации

```
class AdminRegForm(FlaskForm):
    username = StringField('Имя', [validators.InputRequired(), validators.Length(min=4, max=25)])
    password = PasswordField('Пароль', [validators.InputRequired(), validators.Length(min=6, max=100)])
    submit = SubmitField('Зарегистрироваться')
```

форма входа

```
class AdminLoginForm(FlaskForm):
    username = StringField('Логин', [validators.InputRequired()])
    password = PasswordField('Пароль', [validators.InputRequired()])
    remember_me = BooleanField('Запомнить меня')
    submit = SubmitField('Войти')
```

форма выбора параметров расписания

```
class ScheduleForm(FlaskForm):
    group = SelectField('Группа', coerce=int, default=0)
    teacher = SelectField('Преподаватель', coerce=int, default=0)
    aud = SelectField('Корпус и аудитория', coerce=int, default=0)
    submit = SubmitField('Показать')
```

форма добавления потока

```
class InsertStreamForm(FlaskForm):
```

```

dir = SelectField('Направление', coerce=int, default=0)
year = IntegerField('Год поступления', [validators.InputRequired(), validators.NumberRange(min=2000,
max=2100)])
submit = SubmitField('Добавить')

```

форма удаления потока

```

class DeleteStreamForm(FlaskForm):
    stream = SelectField('Поток', coerce=int, default=0)
    submit = SubmitField('Удалить')

```

форма редактирования группы

```

class UpdateGroupForm(FlaskForm):
    group = SelectField('Группа', coerce=int, default=0)
    curr_term = IntegerField('Изменить текущий семестр на', [validators.InputRequired(),
                                                             validators.NumberRange(min=1, max=12)])
    submit = SubmitField('Сохранить')

```

форма добавления группы

```

class InsertGroupForm(FlaskForm):
    group = StringField('Номер группы', [validators.InputRequired()])
    curr_term = IntegerField('Текущий семестр', [validators.InputRequired(),
                                                  validators.NumberRange(min=1, max=12)])
    dir = SelectField('Направление', coerce=int, default=0)
    year = IntegerField('Год поступления', [validators.InputRequired(),
                                             validators.NumberRange(min=2000, max=2100)])
    submit = SubmitField('Добавить')

```

форма удаления группы

```

class DeleteGroupForm(FlaskForm):
    group = SelectField('Группа', coerce=int, default=0)
    submit = SubmitField('Удалить')

```

форма редактирования преподавателя

```

class UpdateTeacherForm(FlaskForm):
    name = SelectField('Преподаватель', coerce=int, default=0)
    new_name = StringField('Новые ФИО')
    position = StringField('Изменить должность на')
    dep_ch = SelectField('Изменить кафедру', coerce=int, choices=[(0, 'не выбрано'), (1, 'изменить'), (2,

```

```
'добавить'), (3, 'удалить')], default=0)
```

```
dep = SelectField('Номер кафедры', coerce=int, default=0)
```

```
submit = SubmitField('Сохранить')
```

форма добавления преподавателя

```
class InsertTeacherForm(FlaskForm):
```

```
    name = StringField('ФИО', [validators.InputRequired()])
```

```
    position = StringField('Должность')
```

```
    rec_num = StringField('Номер трудовой книжки', [validators.InputRequired()])
```

```
    dep = SelectField('Номер кафедры', coerce=int, default=0)
```

```
    submit = SubmitField('Добавить')
```

форма удаления преподавателя

```
class DeleteTeacherForm(FlaskForm):
```

```
    name = SelectField('Преподаватель', coerce=int, default=0)
```

```
    submit = SubmitField('Удалить')
```

форма выбора группы для редактирования расписания

```
class ChooseGroupForm(FlaskForm):
```

```
    group = SelectField('Группа', coerce=int, default=0)
```

```
    submit = SubmitField('Далее')
```

форма редактирования расписания

```
class UpdateScheduleForm(FlaskForm):
```

```
    pare = SelectField('Папа', coerce=int, default=0)
```

```
    wtype = SelectField('Тип недели', coerce=int, choices=[(0, 'не выбрано'), (1, 'Четная'), (2, 'Нечетная')],  
default=0)
```

```
    wday = SelectField('День недели', coerce=int, choices=[(0, 'не выбрано'), (1, 'Понедельник'), (2, 'Вторник'), (3,  
'Среда'), (4, 'Четверг'), (5, 'Пятница'), (6, 'Суббота')], default=0)
```

```
    num = IntegerField('Номер пары', [validators.NumberRange(min=1, max=6), validators.optional()])
```

```
    aud = SelectField('Корпус и аудитория', coerce=int, default=0)
```

```
    course = SelectField('Предмет', coerce=int, default=0)
```

```
    teacher = SelectField('Преподаватель', coerce=int, default=0)
```

```
    submit = SubmitField('Сохранить')
```

форма добавления расписания

```
class InsertScheduleForm(FlaskForm):
```

```
    group = SelectField('Группа', coerce=int, default=0)
```

```

wtype = SelectField('Тип недели', coerce=int, choices=[(0, 'не выбрано'), (1, 'Четная'), (2, 'Нечетная')],
default=0)
wday = SelectField('День недели', coerce=int, choices=[(0, 'не выбрано'), (1, 'Понедельник'), (2, 'Вторник'), (3,
'Среда'), (4, 'Четверг'), (5, 'Пятница'), (6, 'Суббота')], default=0)
num = IntegerField('Номер пары', [validators.NumberRange(min=1, max=6), validators.InputRequired()])
aud = SelectField('Корпус и аудитория', coerce=int, default=0)
course = SelectField('Предмет', coerce=int, default=0)
teacher = SelectField('Преподаватель', coerce=int, default=0)
submit = SubmitField('Добавить')

```

форма удаления расписания

```

class DeleteScheduleForm(FlaskForm):
    group = SelectField('Группа', coerce=int, default=0)
    submit = SubmitField('Удалить')

```

форма редактирования учебного плана

```

class UpdatePlanForm(FlaskForm):
    course = SelectField('Предмет', coerce=int, default=0)
    term = IntegerField('Семестр', [validators.NumberRange(min=1, max=12)])
    lec = IntegerField('Количество лекционных часов', [validators.NumberRange(min=0, max=100),
validators.optional()])
    lab = IntegerField('Количество лабораторных часов', [validators.NumberRange(min=0, max=100),
validators.optional()])
    att = SelectField('Тип аттестации', coerce=int, choices=[(0, 'не выбрано'), (1, 'Экзамен'), (2, 'Зачет'), (3,
'Диф.зачет'), (4, 'Курсовая работа')], default=0)
    submit = SubmitField('Сохранить')

```

форма добавления учебного плана

```

class InsertPlanForm(FlaskForm):
    stream = SelectField('Поток', coerce=int, default=0)
    term = IntegerField('Номер семестра', [validators.NumberRange(min=1, max=12), validators.InputRequired()])
    course = SelectField('Предмет', coerce=int, default=0)
    att = SelectField('Тип аттестации', coerce=int, choices=[(0, 'не выбрано'), (1, 'Экзамен'), (2, 'Зачет'), (3,
'Диф.зачет'), (4, 'Курсовая работа')], default=0)
    lec = IntegerField('Лекционные часы', [validators.NumberRange(min=0, max=100), validators.optional()])
    lab = IntegerField('Лабораторные часы', [validators.NumberRange(min=0, max=100), validators.optional()])
    submit = SubmitField('Добавить')

```

форма удаления учебного плана

```

class DeletePlanForm(FlaskForm):
    course = SelectField('Предмет', coerce=int, default=0)
    submit = SubmitField('Удалить')

# форма добавления предмета
class InsertCourseForm(FlaskForm):
    course = StringField('Название предмета', [validators.InputRequired()])
    submit = SubmitField('Добавить')

# форма редактирования предмета
class UpdateCourseForm(FlaskForm):
    course = SelectField('Название предмета', coerce=int, default=0)
    new_name = StringField('Новое название', [validators.InputRequired()])
    submit = SubmitField('Сохранить')

# форма удаления предмета
class DeleteCourseForm(FlaskForm):
    course = SelectField('Название предмета', coerce=int, default=0)
    submit = SubmitField('Удалить')

```

tupleproc.py

```

# обработка кортежей для select-полей
def tupleProc(input):
    result = [(0, 'не выбрано')]
    result += [(i + 1, item[0]) for i, item in enumerate(input)]
    return result

```

7.2 Модуль регистрации и входа в систему

admin.py

```
from app import app
import psycpg
from app import login_manager
from flask_login import UserMixin

# реализация класса администратора
class Admin(UserMixin):
    def __init__(self, id, username, password):
        self.id = id
        self.username = username
        self.password = password

# функция загрузки пользователя
@login_manager.user_loader
def load_admin(id):
    with psycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        username, password = cur.execute('SELECT username, password '
                                         'FROM admin '
                                         'WHERE id = %s', (id,)).fetchone()
    return Admin(id, username, password)
```

adminRegForm.py

```
from app import app
import psycpg
from app.forms import AdminRegForm
from flask import render_template, flash, redirect, url_for
from flask_login import current_user
from werkzeug.security import generate_password_hash

@app.route('/registration', methods=['GET', 'POST'])
def adminRegForm():
    # если текущий пользователь уже вошел в систему, то перенаправление на главную страницу
    if current_user.is_authenticated:
        return redirect(url_for('index'))
```

```

with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                    password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
    cur = con.cursor()
    admins = cur.execute('SELECT username FROM admin').fetchall()

    # иначе вывод формы регистрации
    form = AdminRegForm()

    # если форма проходит валидацию
    if form.validate_on_submit():
        pwd_hash = generate_password_hash(form.password.data)
        uname = form.username.data
        adm = (uname,)

        if adm in admins:
            flash('Администратор с таким именем существует', 'danger')
            redirect('adminRegForm')

        else:
            with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                                password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
                cur = con.cursor()
                cur.execute('INSERT INTO admin(username, password) '
                            'VALUES (%s, %s)', (uname, pwd_hash))

            flash('Регистрация прошла успешно', 'success')
            return redirect(url_for('index'))

    return render_template('adminRegForm.html', title='Регистрация', form=form)

```

adminLoginForm.py

```

from werkzeug.security import check_password_hash
from app import app
import psycopg
from app.forms import AdminLoginForm
from app.admin import Admin
from flask import render_template, redirect, flash, url_for
from flask_login import login_user, current_user

```

```

@app.route('/login', methods=['GET', 'POST'])
def adminLoginForm():
    # если текущий пользователь уже вошел в систему, то перенаправление на главную страницу
    if current_user.is_authenticated:
        return redirect(url_for('index'))

    # иначе вывод формы входа
    form = AdminLoginForm()

    # если форма проходит валидацию
    if form.validate_on_submit():
        # подключение к базе данных
        with psycopg2.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                               password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
            cur = con.cursor()
            # находим запись в базе с соответствующим именем пользователя
            res = cur.execute('SELECT id, username, password FROM admin '
                              'WHERE username = %s', (form.username.data,)).fetchone()

            # если пользователь не найден или пароле не соответствует
            if res is None or not check_password_hash(res[2], form.password.data):
                flash('Неверный логин или пароль', 'danger')
                return redirect(url_for('adminLoginForm'))

            id, username, password = res
            admin = Admin(id, username, password)

            # создание сессии и запись в нее аутентификационных данных
            login_user(admin, remember=form.remember_me.data)

            flash('Вы успешно вошли в систему', 'success')
            return redirect(url_for('index'))

    return render_template('adminLoginForm.html', title='Вход', form=form)

```

adminLogout.py

```
from app import app
from flask_login import logout_user
from flask import redirect, url_for

@app.route('/logout')
def adminLogout():
    # выход из системы
    logout_user()
    return redirect(url_for('index'))
```

7.3 Модуль списка и редактирования предмета

courses.py

```
from app import app
from flask import render_template
import psycopg

@app.get('/courses')
def courses():
    # подключение к базе данных и получения списка предметов
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        courses = cur.execute('SELECT * FROM course ORDER BY name').fetchall()

    return render_template('courses.html', title='Предметы', courses=courses)
```

editCourse.py

```
from app import app
from flask import render_template
from flask_login import login_required

@app.get('/edit/course')
@login_required
def editCourse():
    return render_template('editCourse.html', title='Управление предметами')
```

updateCourseForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import UpdateCourseForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/course/update', methods=['GET', 'POST'])
@login_required
def updateCourseForm():
```

```

# подключение к базе данных и получение списка предметов
with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                     password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
    cur = con.cursor()
    courses = tupleProc(cur.execute('SELECT * FROM course ORDER BY name').fetchall())

if courses is None:
    abort(404)

# форма редактирования предмета
form = UpdateCourseForm()
form.course.choices = courses

# если форма проходит валидацию
if form.validate_on_submit():
    new_name = form.new_name.data
    course = dict(form.course.choices).get(int(form.course.data))

    # если предмет не выбран
    if form.course.data == 0:
        flash('Выберите предмет', 'danger')
        redirect(url_for('updateCourseForm'))

    # или такой предмет уже существует
    elif new_name in courses:
        flash('Предмет с таким названием уже существует', 'danger')
        redirect(url_for('updateCourseForm'))

    else:
        with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                           password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
            cur = con.cursor()
            # добавление предмета
            cur.execute('UPDATE course '
                      'SET name = %s '
                      'WHERE name = %s', (new_name, course,))

```

```
flash('Данные успешно обновлены', 'success')
redirect(url_for('updateCourseForm'))

return render_template('renderForm.html', title='Редактирование существующего предмета', form=form)
```

insertCourseForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for
import pycpg
from app.forms import InsertCourseForm
from flask_login import login_required

@app.route('/edit/course/insert', methods=['GET', 'POST'])
@login_required
def insertCourseForm():
    # подключение к базе данных и получение списка предметов
    with pycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                      password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        courses = cur.execute('SELECT * FROM course').fetchall()

    # форма добавления предмета
    form = InsertCourseForm()

    # если форма проходит валидацию
    if form.validate_on_submit():
        course = form.course.data
        c = (course,)

        # если такой предмет уже существует
        if courses is not None and c in courses:
            flash('Такой предмет уже существует', 'danger')
            redirect(url_for('insertCourseForm'))

    else:
        with pycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                          password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
```

```

        cur = con.cursor()
        # добавление предмета
        cur.execute('INSERT INTO course VALUES (%s)', (course,))

    flash('Данные успешно обновлены', 'success')
    redirect(url_for('insertCourseForm'))

return render_template('renderForm.html', title='Добавление нового предмета', form=form)

```

deleteCourseForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import DeleteCourseForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/course/delete', methods=['GET', 'POST'])
@login_required
def deleteCourseForm():
    # подключение к базе данных и получение списка предметов
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        courses = tupleProc(cur.execute('SELECT * FROM course ORDER BY name').fetchall())

    if courses is None:
        abort(404)

    # форма удаления предмета
    form = DeleteCourseForm()
    form.course.choices = courses

    # если форма проходит валидацию
    if form.validate_on_submit():
        course = dict(form.course.choices).get(int(form.course.data))

```

```

# если предмет не выбран
if form.course.data == 0:
    flash('Выберите предмет', 'danger')
    redirect(url_for('deleteCourseForm'))

else:
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        # удаление предмета
        cur.execute('DELETE FROM course '
                    'WHERE name = %s', (course,))

    flash('Данные успешно обновлены', 'success')
    redirect(url_for('deleteCourseForm'))

flash('При удалении предмета будут удалены связанные с ним пары и учебные планы!', 'warning')
return render_template('renderForm.html', title='Удаление существующего предмета', form=form)

```

7.4 Модуль просмотра и редактирования учебного плана

edPlanShow.py

```
from app import app
import psycopg
from flask import render_template

@app.get('/directions/edplan/<dir>/<year>')
def edPlanShow(dir, year):
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        plan = cur.execute('SELECT term_number, course_name, att_type, lec_hours, lab_hours '
                            'FROM term_stream_by_course '
                            'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                            'ORDER BY stream_dir_code, stream_entry_year, term_number', (dir, year,)).fetchall()

    return render_template('edPlanShow.html', title=f'Учебный план направления {dir} на {year} год', plan=plan)
```

editPlan.py

```
from app import app
from flask import render_template
from flask_login import login_required

@app.get('/edit/edplan')
@login_required
def editEdPlan():
    return render_template('editEdplan.html', title='Управление учебным планом')
```

updateEdplan.py

```
from app import app
from flask import render_template
import psycopg
from flask_login import login_required

@app.get('/edit/edplan/update')
@login_required
```

```
def updateEdplan():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        streams = cur.execute('SELECT stream_dir_code, entry_year FROM stream '
                              'ORDER BY stream_dir_code, entry_year').fetchall()

    return render_template('updateEdplan.html', title='Учебные планы потоков', streams=streams)
```

updatePlanForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import UpdatePlanForm
from flask_login import login_required

@app.route('/edit/edplan/update/<dir>/<year>', methods=['GET', 'POST'])
@login_required
def updatePlanForm(dir, year):
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'],
                        dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        plans = cur.execute('SELECT course_name, term_number '
                              'FROM term_stream_by_course '
                              'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                              'ORDER BY term_number, course_name', (dir, year,)).fetchall()

    if plans is None:
        abort(404)

    # обработка строк кортежа plans
    result = [(0, 'не выбрано')]
    result += [(i + 1, f'{item[0]}, {item[1]} семестр') for i, item in enumerate(plans)]

    form = UpdatePlanForm()
    form.course.choices = result
```

```

if form.validate_on_submit():
    if form.course.data == 0:
        flash('Выберите план', 'danger')
        redirect(url_for('updatePlanForm', dir=dir, year=year))

    elif form.lec.data is None and form.lab.data is None and form.term.data is None and form.att.data == 0:
        flash('Заполните хотя бы одно поле', 'danger')
        redirect(url_for('updatePlanForm', dir=dir, year=year))

    else:
        pl = dict(form.course.choices).get(int(form.course.data))
        p = pl.split(", ")
        course = p[0]
        term = p[1][0]
        new_term = form.term.data or None
        lec = form.lec.data or None
        lab = form.lab.data or None
        att = dict(form.att.choices).get(int(form.att.data))

        with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                             password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
            cur = con.cursor()

            if lec is not None:
                cur.execute('UPDATE term_stream_by_course '
                            'SET lec_hours = %s '
                            'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                            'AND course_name = %s AND term_number = %s', (lec, dir, year, course, term,))

            if lab is not None:
                cur.execute('UPDATE term_stream_by_course '
                            'SET lab_hours = %s '
                            'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                            'AND course_name = %s AND term_number = %s', (lab, dir, year, course, term,))

            if att != "не выбрано":
                cur.execute('UPDATE term_stream_by_course '

```

```

        'SET att_type = %s '
        'WHERE stream_dir_code = %s AND stream_entry_year = %s '
        'AND course_name = %s AND term_number = %s', (att, dir, year, course, term,))

    if new_term is not None:
        cur.execute('UPDATE term_stream_by_course '
                    'SET term_number = %s '
                    'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                    'AND course_name = %s AND term_number = %s', (new_term, dir, year, course, term,))

    flash('Данные успешно обновлены', 'success')
    redirect(url_for('updatePlanForm', dir=dir, year=year))

return render_template('renderForm.html', title='Редактирование существующего учебного плана', form=form)

```

insertPlanForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycpg
from app.forms import InsertPlanForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/edplan/insert', methods=['GET', 'POST'])
@login_required
def insertPlanForm():
    with psycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        streams = cur.execute('SELECT stream_dir_code, entry_year FROM stream ORDER BY stream_dir_code,
entry_year').fetchall()
        courses = tupleProc(cur.execute('SELECT * FROM course ORDER BY name').fetchall())
        plans = cur.execute('SELECT term_number, stream_dir_code, stream_entry_year, course_name '
                              'FROM term_stream_by_course').fetchall()

    if streams is None or courses is None:
        abort(404)

```

```

result = [(0, 'не выбрано' + ")]
result += [(i + 1, f"{item[0]}, {item[1]}") for i, item in enumerate(streams)]

form = InsertPlanForm()
form.stream.choices = result
form.course.choices = courses

if form.validate_on_submit():
    st = dict(form.stream.choices).get(int(form.stream.data))
    s = st.split(", ")
    dir = s[0]
    year = s[1]
    course = dict(form.course.choices).get(int(form.course.data))
    term = form.term.data
    att = dict(form.att.choices).get(int(form.att.data)) or None
    lec = form.lec.data or None
    lab = form.lab.data or None

    pl = (term, dir, int(year), course,)

    if form.stream.data == 0 or form.course.data == 0:
        flash('Выберите поток и предмет', 'danger')
        redirect(url_for('insertPlanForm'))

    elif plans is not None and pl in plans:
        flash('Семестр по этому предмету с данными параметрами уже существует', 'danger')
        redirect(url_for('insertPlanForm'))

    else:
        with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                             password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
            cur = con.cursor()
            cur.execute('INSERT INTO term_stream_by_course (term_number, stream_dir_code, stream_entry_year,
course_name)'
                        'VALUES (%s, %s, %s, %s)', (term, dir, year, course))

```

```

# если указан тип аттестации
if att is not None:
    cur.execute('UPDATE term_stream_by_course '
                'SET att_type = %s '
                'WHERE term_number = %s AND stream_dir_code = %s '
                'AND stream_entry_year = %s AND course_name = %s', (att, term, dir, year, course))

# если указаны лекционные часы
if lec is not None:
    cur.execute('UPDATE term_stream_by_course '
                'SET lec_hours = %s '
                'WHERE term_number = %s AND stream_dir_code = %s '
                'AND stream_entry_year = %s AND course_name = %s', (lec, term, dir, year, course))

# если указаны лабораторные часы
if lab is not None:
    cur.execute('UPDATE term_stream_by_course '
                'SET lab_hours = %s '
                'WHERE term_number = %s AND stream_dir_code = %s '
                'AND stream_entry_year = %s AND course_name = %s', (lab, term, dir, year, course))

flash('Данные успешно обновлены', 'success')
redirect(url_for('insertPlanForm'))

return render_template('renderForm.html', title='Добавление нового учебного плана', form=form)

```

deleteEdplan.py

```

from app import app
from flask import render_template
import psycopg

@app.get('/edit/edplan/delete')
def deleteEdplan():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        streams = cur.execute('SELECT stream_dir_code, entry_year '

```

```

        'FROM stream '
        'ORDER BY stream_dir_code, entry_year').fetchall()

return render_template('deleteEdplan.html', title='Выберите поток', streams=streams)

```

deletePlanForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import DeletePlanForm
from flask_login import login_required

@app.route('/edit/edplan/delete/<dir>/<year>', methods=['GET', 'POST'])
@login_required
def deletePlanForm(dir, year):
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        plans = cur.execute('SELECT course_name, term_number '
                            'FROM term_stream_by_course '
                            'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                            'ORDER BY term_number, course_name', (dir, year,)).fetchall()

    if plans is None:
        abort(404)

    # обработка строк кортежа plans
    result = [(0, 'не выбрано')]
    result += [(i + 1, f"{item[0]}, {item[1]} семестр") for i, item in enumerate(plans)]

    form = DeletePlanForm()
    form.course.choices = result

    if form.validate_on_submit():
        if form.course.data == 0:
            flash('Выберите план', 'danger')
            redirect(url_for('deletePlanForm', dir=dir, year=year))

```

```

else:
    pl = dict(form.course.choices).get(int(form.course.data))
    p = pl.split(", ")
    course = p[0]
    term = p[1][0]

    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        cur.execute('DELETE FROM term_stream_by_course '
                    'WHERE stream_dir_code = %s AND stream_entry_year = %s '
                    'AND course_name = %s AND term_number = %s', (dir, year, course, term,))

    flash('Данные успешно обновлены', 'success')
    redirect(url_for('deletePlanForm', dir=dir, year=year))

return render_template('renderForm.html', title='Удаление существующего учебного плана', form=form)

```

7.5 Модуль просмотра и редактирования групп

groups.py

```
from app import app
from flask import render_template
import psycopg

@app.get('/groups')
def groups():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = cur.execute('SELECT group_number, curr_term, group_dir_code, entry_year, direction_dep_number '
                              'FROM group_tb JOIN stream ON group_dir_code = stream_dir_code AND group_entry_year = '
                              'entry_year '
                              'JOIN direction ON stream_dir_code = code '
                              'JOIN direction_department ON code = dir_code '
                              'ORDER BY group_number').fetchall()

    return render_template('groups.html', title='Группы', groups=groups)
```

editGroup.py

```
from app import app
from flask import render_template
from flask_login import login_required

@app.get('/edit/group')
@login_required
def editGroup():
    return render_template('editGroup.html', title='Управление группами')
```

updateGroupForm.py

```
from app import app
from app.forms import UpdateGroupForm
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.tupleproc import tupleProc
from flask_login import login_required
```

```

@app.route('/edit/group/update', methods=['GET', 'POST'])
@login_required
def updateGroupForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = tupleProc(cur.execute('SELECT group_number FROM group_tb ORDER BY
group_number')).fetchall()

    if groups is None:
        abort(404)

    form = UpdateGroupForm()
    form.group.choices = groups

    if form.validate_on_submit():
        if form.group.data == 0:
            flash('Выберите группу', 'danger')
            redirect(url_for('updateGroupForm'))

        else:
            gr = dict(form.group.choices).get(int(form.group.data))
            new_term = form.curr_term.data

            with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                                password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
                cur = con.cursor()
                cur.execute('UPDATE group_tb '
                            'SET curr_term = %s '
                            'WHERE group_number = %s', (new_term, gr,))

            flash('Данные успешно обновлены', 'success')
            redirect(url_for('updateGroupForm'))

    return render_template('renderForm.html', title='Редактирование существующей группы', form=form)

```

insertGroupForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import InsertGroupForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/group/insert', methods=['GET', 'POST'])
@login_required
def insertGroupForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        directions = tupleProc(cur.execute('SELECT code FROM direction ORDER BY code').fetchall())
        groups = cur.execute('SELECT group_number FROM group_tb').fetchall()
        streams = cur.execute('SELECT * FROM stream').fetchall()

    if directions is None:
        abort(404)

    form = InsertGroupForm()
    form.dir.choices = directions

    if form.validate_on_submit():
        gr = (form.group.data,)
        term = form.curr_term.data
        dir = dict(form.dir.choices).get(int(form.dir.data))
        year = form.year.data
        st = (year, dir)

        # если группа не выбрана
        if form.dir.data == 0:
            flash('Выберите направление', 'danger')
            redirect(url_for('insertGroupForm'))

        # если такая группа уже существует
```

```

elif groups is not None and gr in groups:
    flash('Группа с таким номером уже существует', 'danger')
    redirect(url_for('insertGroupForm'))

# если не существует такого потока

elif streams is not None and st not in streams:
    flash(f'Не существует потока {year} года направления {dir}', 'danger')
    redirect(url_for('insertGroupForm'))

else:
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        cur.execute('INSERT INTO group_tb VALUES (%s, %s, %s, %s)', (form.group.data, term, dir, year,))

    flash('Данные успешно обновлены', 'success')
    redirect(url_for('insertGroupForm'))

return render_template('insertGroupForm.html', title='Добавление новой группы', form=form)

```

deleteGroupForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import DeleteGroupForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/group/delete', methods=['GET', 'POST'])
@login_required
def deleteGroupForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = tupleProc(cur.execute('SELECT group_number FROM group_tb ORDER BY
group_number').fetchall())

```

```

if groups is None:
    abort(404)

form = DeleteGroupForm()
form.group.choices = groups

if form.validate_on_submit():
    if form.group.data == 0:
        flash('Выберите группу', 'danger')
        redirect(url_for('deleteGroupForm'))
    else:
        gr = dict(form.group.choices).get(int(form.group.data))
        with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                             password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
            cur = con.cursor()
            cur.execute('DELETE FROM class '
                        'WHERE (week_day, class_number, week_type, room_number, building_address) IN '
                        '('
                        'SELECT week_day, class_number, week_type, room_number, building_address '
                        'FROM group_class '
                        'WHERE group_number = %s'
                        ')', (gr,))

            cur.execute('DELETE FROM group_tb '
                        'WHERE group_number = %s', (gr,))

        flash('Данные успешно обновлены', 'success')
        redirect(url_for('deleteGroupForm'))

flash('При удалении группы будет удалено связанное с ней расписание!', 'warning')
return render_template('renderForm.html', title='Удаление существующей группы', form=form)

```

7.6 Модуль просмотра и редактирования расписания

scheduleForm.py

```
from app import app
import psycopg
from app.forms import ScheduleForm
from flask import render_template, flash, redirect, url_for, abort
from app.tupleproc import tupleProc

@app.route('/schedule', methods=['GET', 'POST'])
def scheduleForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
        password=app.config['DB_PASSWORD'],
            dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = tupleProc(cur.execute('SELECT group_number, group_name FROM group_tb ORDER BY
group_number').fetchall())
        teachers = tupleProc(cur.execute('SELECT full_name FROM teacher ORDER BY full_name').fetchall())
        auds = cur.execute('SELECT building_address, room_number FROM classroom').fetchall()

    if groups is None or teachers is None or auds is None:
        abort(404)

    # обработка строк кортежа auds
    rooms = [(0, 'не выбрано')]
    rooms += [(i + 1, f"{item[0]}, ауд. {item[1]}") for i, item in enumerate(auds)]

    form = ScheduleForm()
    form.group.choices = groups
    form.teacher.choices = teachers
    form.aud.choices = rooms

    if form.validate_on_submit():
        if (form.group.data == 0 and form.teacher.data == 0 and form.aud.data == 0):
            flash('Выберите хотя бы один параметр', 'warning')
            return redirect(url_for('scheduleForm'))

        else:
```

```

group = dict(form.group.choices).get(int(form.group.data))
teacher = dict(form.teacher.choices).get(int(form.teacher.data))

if form.aud.data != 0:
    room = dict(form.aud.choices).get(int(form.aud.data))
    r = room.split(", ауд. ")
    corp = r[0]
    aud = r[1]
else:
    corp = 'не выбрано'
    aud = 'не выбрано'

return redirect(url_for('scheduleShow', group=group, teacher=teacher, corp=corp, aud=aud))

return render_template('renderForm.html', title='Параметры расписания', form=form)

```

scheduleShow.py

```

from app import app
import psycpg
from flask import render_template

@app.route('/schedule/<group>/<teacher>/<corp>/<aud>', methods=['GET', 'POST'])
def scheduleShow(group, teacher, corp, aud):
    with psycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()

        if group == 'не выбрано' and teacher == 'не выбрано' and corp != 'не выбрано' and aud != 'не выбрано':
            title = f"Расписание для корпус {corp}, ауд. {aud}"
            rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
                                full_name, building_address, room_number '
                                'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,
                                room_number) '
                                'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
                                'JOIN day_of_week ON week_day = day_of_week.day '
                                'WHERE building_address = %s AND room_number = %s '
                                'ORDER BY week_type, day_number', (corp, aud,)).fetchall()

```

```

elif group == 'не выбрано' and teacher != 'не выбрано' and corp == 'не выбрано' and aud == 'не выбрано':
    title = f"Расписание для преподавателя {teacher}"
    rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
full_name, building_address, room_number '
                        'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,
room_number) '
                        'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
                        'JOIN day_of_week ON week_day = day_of_week.day '
                        'WHERE full_name = %s '
                        'ORDER BY week_type, day_number', (teacher,)).fetchall()

```

```

elif group == 'не выбрано' and teacher != 'не выбрано' and corp != 'не выбрано' and aud != 'не выбрано':
    title = f"Расписание для преподавателя {teacher}, корпус {corp}, ауд. {aud}"
    rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
full_name, building_address, room_number '
                        'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,
room_number) '
                        'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
                        'JOIN day_of_week ON week_day = day_of_week.day '
                        'WHERE full_name = %s AND building_address = %s AND room_number = %s '
                        'ORDER BY week_type, day_number', (teacher, corp, aud,)).fetchall()

```

```

elif group != 'не выбрано' and teacher == 'не выбрано' and corp == 'не выбрано' and aud == 'не выбрано':
    title = f"Расписание для группы {group}"
    rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
full_name, building_address, room_number '
                        'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,
room_number) '
                        'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
                        'JOIN day_of_week ON week_day = day_of_week.day '
                        'WHERE group_number = %s '
                        'ORDER BY week_type, day_number', (group,)).fetchall()

```

```

elif group != 'не выбрано' and teacher == 'не выбрано' and corp != 'не выбрано' and aud != 'не выбрано':
    title = f"Расписание для группы {group}, корпус {corp}, ауд. {aud}"
    rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
full_name, building_address, room_number '
                        'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,

```

```

room_number) '
    'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
    'JOIN day_of_week ON week_day = day_of_week.day '
    'WHERE group_number = %s AND building_address = %s AND room_number = %s '
    'ORDER BY week_type, day_number', (group, corp, aud,)).fetchall()

elif group != 'не выбрано' and teacher != 'не выбрано' and corp == 'не выбрано' and aud == 'не выбрано':
    title = f"Расписание для группы {group}, преподавателя {teacher}"
    rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
full_name, building_address, room_number '
        'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,
room_number) '
        'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
        'JOIN day_of_week ON week_day = day_of_week.day '
        'WHERE group_number = %s AND full_name = %s '
        'ORDER BY week_type, day_number', (group, teacher,)).fetchall()

elif group != 'не выбрано' and teacher != 'не выбрано' and corp != 'не выбрано' and aud != 'не выбрано':
    title = f"Расписание для группы {group}, преподавателя {teacher}, корпус {corp}, ауд. {aud}"
    rasp = cur.execute('SELECT group_number, week_type, week_day, class_number, class_course_name,
full_name, building_address, room_number '
        'FROM group_class JOIN class USING (week_day, class_number, week_type, building_address,
room_number) '
        'JOIN teacher ON class.teacher_emp_record_num = teacher.emp_record_num '
        'JOIN day_of_week ON week_day = day_of_week.day '
        'WHERE group_number = %s AND full_name = %s AND building_address = %s AND
room_number = %s '
        'ORDER BY week_type, day_number', (group, teacher, corp, aud,)).fetchall()

return render_template('scheduleShow.html', title=title, rasp=rasp)

```

editSchedule.py

```

from app import app
from flask import render_template
from flask_login import login_required

@app.get('/edit/schedule')
@login_required

```

```
def editSchedule():
    return render_template('editSchedule.html', title='Управление расписанием')
```

updateSchedule.py

```
from app import app
from flask import render_template
import psycopg
from flask_login import login_required

@app.route('/edit/schedule/update', methods=['GET', 'POST'])
@login_required
def chooseGroupForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = cur.execute('SELECT group_number FROM group_tb ORDER BY group_number').fetchall()

    return render_template('updateSchedule.html', title='Редактирование расписания группы', groups=groups)
```

updateSheduleForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import UpdateScheduleForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/schedule/update/<group>', methods=['GET', 'POST'])
@login_required
def updateScheduleForm(group):
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'],
                        dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        pare = cur.execute('SELECT week_type, week_day, class_number, class_course_name, '
                            'building_address, room_number, full_name '
                            'FROM group_class JOIN class '
                            'USING (week_day, class_number, week_type, room_number, building_address) ')
```

```

        'JOIN teacher ON teacher_emp_record_num = emp_record_num '
        'JOIN day_of_week ON day = week_day '
        'WHERE group_number = %s '
        'ORDER BY week_type, day_number', (group,)).fetchall()
courses = tupleProc(cur.execute('SELECT * FROM course ORDER BY name').fetchall())
teachers = tupleProc(cur.execute('SELECT full_name FROM teacher ORDER BY full_name').fetchall())
auds = cur.execute('SELECT building_address, room_number FROM classroom').fetchall()

if pare is None or auds is None or courses is None or teachers is None:
    abort(404)

rooms = [(0, 'не выбрано')]
rooms += [(i + 1, f'{item[0]}, ауд. {item[1]}') for i, item in enumerate(auds)]

pares = [(0, 'не выбрано')]
pares += [(i + 1, f'{item[0]} неделя, {item[1]}, {item[2]} пара, {item[3]}, {item[4]}, {item[5]}, {item[6]}') for
i, item in enumerate(pare)]

form = UpdateScheduleForm()
form.pare.choices = pares
form.aud.choices = rooms
form.course.choices = courses
form.teacher.choices = teachers

if form.validate_on_submit():
    if form.pare.data == 0:
        flash('Выберите пару', 'danger')
        redirect(url_for('updateScheduleForm', group=group))

    elif form.wtype.data == 0 and form.wday.data == 0 and form.num.data is None and form.aud.data == 0 and
form.course.data == 0 and form.teacher.data == 0:
        flash('Заполните хотя бы одно поле', 'danger')

    else:
        wtype = dict(form.wtype.choices).get(int(form.wtype.data))
        wday = dict(form.wday.choices).get(int(form.wday.data))
        num = form.num.data or None

```

```

course = dict(form.course.choices).get(int(form.course.data))
teacher = dict(form.teacher.choices).get(int(form.teacher.data))
if form.aud.data != 0:
    room = dict(form.aud.choices).get(int(form.aud.data))
    r = room.split(", ауд. ")
    corp = r[0]
    aud = r[1]

# ключи
ktype = pare[0][0]
kday = pare[0][1]
knum = pare[0][2]
kaddr = pare[0][4]
kroom = pare[0][5]

with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                      password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
    cur = con.cursor()

    if course != 'не выбрано':
        cur.execute('UPDATE class '
                    'SET class_course_name = %s '
                    'WHERE week_type = %s AND week_day = %s AND class_number = %s '
                    'AND building_address = %s AND room_number = %s', (course, ktype, kday, knum, kaddr,
kroom))

    if teacher != 'не выбрано':
        cur.execute('UPDATE class '
                    'SET teacher_emp_record_num = '
                    '('
                    'SELECT emp_record_num '
                    'FROM teacher '
                    'WHERE full_name = %s'
                    ')'
                    'WHERE week_type = %s AND week_day = %s AND class_number = %s '
                    'AND building_address = %s AND room_number = %s', (teacher, ktype, kday, knum, kaddr,
kroom))

```

```

if wtype != 'не выбрано':
    cur.execute('UPDATE class '
                'SET week_type = %s '
                'WHERE week_type = %s AND week_day = %s AND class_number = %s '
                'AND building_address = %s AND room_number = %s',
                (wtype, ktype, kday, knum, kaddr, kroom))
    ktype = wtype

if wday != 'не выбрано':
    cur.execute('UPDATE class '
                'SET week_day = %s '
                'WHERE week_type = %s AND week_day = %s AND class_number = %s '
                'AND building_address = %s AND room_number = %s',
                (wday, ktype, kday, knum, kaddr, kroom))
    kday = wday

if num is not None:
    cur.execute('UPDATE class '
                'SET class_number = %s '
                'WHERE week_type = %s AND week_day = %s AND class_number = %s '
                'AND building_address = %s AND room_number = %s',
                (num, ktype, kday, knum, kaddr, kroom))
    knum = num

if form.aud.data != 0:
    cur.execute('UPDATE class '
                'SET building_address = %s, room_number = %s '
                'WHERE week_type = %s AND week_day = %s AND class_number = %s '
                'AND building_address = %s AND room_number = %s',
                (corp, aud, ktype, kday, knum, kaddr, kroom))
    kaddr = corp
    kroom = aud

flash('Данные успешно обновлены', 'success')
redirect(url_for('updateScheduleForm', group=group))

return render_template('renderForm.html', title='Редактирование существующего расписания', form=form)

```

insertSheduleForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import InsertScheduleForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/schedule/insert', methods=['GET', 'POST'])
@login_required
def insertScheduleForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
        password=app.config['DB_PASSWORD'],
            dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = tupleProc(cur.execute('SELECT group_number FROM group_tb ORDER BY
group_number')).fetchall()
        auds = cur.execute('SELECT building_address, room_number FROM classroom').fetchall()
        courses = tupleProc(cur.execute('SELECT name FROM course ORDER BY name')).fetchall()
        teachers = tupleProc(cur.execute('SELECT full_name FROM teacher ORDER BY full_name')).fetchall()
        schedule = cur.execute('SELECT group_number, week_type, week_day, class_number, '
            'building_address, room_number, class_course_name, full_name '
            'FROM group_class JOIN class '
            'USING (week_day, class_number, week_type, room_number, building_address) '
            'JOIN teacher ON emp_record_num = teacher_emp_record_num'). fetchall()

    if groups is None or auds is None or courses is None or teachers is None:
        abort(404)

    rooms = [(0, 'не выбрано')]
    rooms += [(i + 1, f"{item[0]}, ауд. {item[1]}") for i, item in enumerate(auds)]

    form = InsertScheduleForm()
    form.group.choices = groups
    form.aud.choices = rooms
    form.course.choices = courses
    form.teacher.choices = teachers
```

```

if form.validate_on_submit():
    if form.group.data == 0 or form.wtype.data == 0 or form.wday.data == 0 or form.aud.data == 0 or
form.course.data == 0 or form.teacher.data == 0:
        flash('Все поля должны быть заполнены', 'danger')
        return redirect(url_for('insertScheduleForm'))

    else:
        gr = dict(form.group.choices).get(int(form.group.data))
        wtype = dict(form.wtype.choices).get(int(form.wtype.data))
        wday = dict(form.wday.choices).get(int(form.wday.data))
        num = form.num.data
        course = dict(form.course.choices).get(int(form.course.data))
        teacher = dict(form.teacher.choices).get(int(form.teacher.data))
        if form.aud.data != 0:
            room = dict(form.aud.choices).get(int(form.aud.data))
            r = room.split(", ауд. ")
            corp = r[0]
            aud = r[1]

        sch = (gr, wtype, wday, num, corp, aud, course, teacher,)

        if schedule is not None and sch in schedule:
            flash('Папа с такими параметрами уже существует', 'danger')
            return redirect(url_for('insertScheduleForm'))

        else:
            with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                                password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
                cur = con.cursor()
                emp_rec = cur.execute('SELECT emp_record_num FROM teacher WHERE full_name = %s',
                                     (teacher,)).fetchone()
                cur.execute('INSERT INTO class VALUES (%s, %s, %s, %s, %s, %s, %s)', (wday, num, wtype, aud,
                                     corp, course, emp_rec[0]))
                cur.execute('INSERT INTO group_class VALUES (%s, %s, %s, %s, %s, %s)', (gr, wday, num, wtype,
                                     aud, corp))

            flash('Данные успешно обновлены', 'success')
            redirect(url_for('insertScheduleForm'))

```

```
return render_template('renderForm.html', title='Добавление нового расписания', form=form)
```

deleteScheduleForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import DeleteScheduleForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/schedule/delete', methods=['GET', 'POST'])
@login_required
def deleteScheduleForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        groups = tupleProc(cur.execute('SELECT group_number FROM group_tb ORDER BY
group_number')).fetchall()

    if groups is None:
        abort(404)

    form = DeleteScheduleForm()
    form.group.choices = groups

    if form.validate_on_submit():
        if form.group.data == 0:
            flash('Выберите группу', 'danger')
            redirect(url_for('deleteGroupForm'))

        else:
            gr = dict(form.group.choices).get(int(form.group.data))

            with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                                password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
                cur = con.cursor()
```

```

cur.execute('DELETE FROM class '
            'WHERE (week_day, class_number, week_type, room_number, building_address) IN '
            '('
            'SELECT week_day, class_number, week_type, room_number, building_address '
            'FROM group_class '
            'WHERE group_number = %s'
            ')', (gr,))

flash('Данные успешно обновлены', 'success')
redirect(url_for('deleteScheduleForm'))

return render_template('renderForm.html', title='Удаление существующего расписания', form=form)

```

7.7 Модуль просмотра направлений и редактирования потоков

directions.py

```
from app import app
from flask import render_template
import psycopg
from collections import defaultdict

@app.get('/directions')
def directions():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        directions = cur.execute('SELECT code, name, entry_year '
                                'FROM direction JOIN stream ON code = stream_dir_code '
                                'ORDER BY code, entry_year').fetchall()

        grouped = defaultdict(list)
        for i in range(len(directions)):
            key = (directions[i][0], directions[i][1])
            grouped[key].append(directions[i][2])

    return render_template('directions.html', title='Направления', directions=grouped)
```

editStream.py

```
from app import app
from flask import render_template
from flask_login import login_required

@app.get('/edit/stream')
@login_required
def editStream():
    return render_template('editStream.html', title='Управление потоками')
```

insertStreamForm.py

```
from app import app
from flask import render_template, flash, redirect, url_for, abort
```

```

import psycpg
from app.forms import InsertStreamForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/stream/insert', methods=['GET', 'POST'])
@login_required
def insertStreamForm():
    with psycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        directions = tupleProc(cur.execute('SELECT code FROM direction ORDER BY code').fetchall())
        streams = cur.execute('SELECT * FROM stream').fetchall()

    if directions is None:
        abort(404)

    form = InsertStreamForm()
    form.dir.choices = directions

    if form.validate_on_submit():
        st = (form.year.data, dict(form.dir.choices).get(int(form.dir.data)),)
        dir = dict(form.dir.choices).get(int(form.dir.data))
        year = form.year.data

        if form.dir.data == 0:
            flash('Выберите направление', 'danger')
            redirect(url_for('insertStreamForm'))

        elif streams is not None and st in streams:
            flash('Такой поток уже существует', 'danger')
            redirect(url_for('insertStreamForm'))

        else:
            with psycpg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                                password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
                cur = con.cursor()

```

```

cur.execute('INSERT INTO stream VALUES (%s, %s)', (year, dir,))

flash('Данные успешно обновлены', 'success')
redirect(url_for('insertStreamForm'))

return render_template('renderForm.html', title='Добавление нового потока', form=form)

```

deleteStreamForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import DeleteStreamForm
from flask_login import login_required

@app.route('/edit/stream/delete', methods=['GET', 'POST'])
@login_required
def deleteStreamForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        streams = cur.execute('SELECT stream_dir_code, entry_year FROM stream ORDER BY stream_dir_code,
entry_year').fetchall()

    if streams is None:
        abort(404)

    result = [(0, 'не выбрано' + ")]
    result += [(i + 1, f'{item[0]}, {item[1]}') for i, item in enumerate(streams)]

    form = DeleteStreamForm()
    form.stream.choices = result

    if form.validate_on_submit():
        if form.stream.data == 0:
            flash('Выберите поток', 'danger')
            redirect(url_for('deleteStreamForm'))
        else:

```

```

st = dict(form.stream.choices).get(int(form.stream.data))
s = st.split(", ")
dir = s[0]
year = s[1]

with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                    password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
    cur = con.cursor()
    cur.execute('DELETE FROM class '
                'WHERE (week_day, class_number, week_type, room_number, building_address) IN '
                '('
                'SELECT week_day, class_number, week_type, room_number, building_address '
                'FROM group_class JOIN group_tb USING (group_number) '
                'WHERE group_dir_code = %s AND group_entry_year = %s'
                ')', (dir, year))

    cur.execute('DELETE FROM stream '
                'WHERE stream_dir_code = %s AND entry_year = %s', (dir, year,))

    flash('Данные успешно обновлены', 'success')
    redirect(url_for('deleteGroupForm'))

flash('При удалении потока будут удалены связанные с ним группы и их расписание!', 'warning')
return render_template('renderForm.html', title='Удаление существующего потока', form=form)

```

7.8 Модуль просмотра и редактирования преподавателей

teachers.py

```
from app import app
from flask import render_template
import psycopg
from collections import defaultdict

@app.get('/teachers')
def teachers():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'],
                        dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        teachers = cur.execute('SELECT full_name, position, teacher_dep_number FROM teacher JOIN
teacher_department '
                                'ON emp_record_num = teacher_emp_record_num ORDER BY full_name').fetchall()

    # обработка кортежа teachers, чтобы при наличии у одного преподавателя нескольких кафедр, не
повторялись ФИО
    grouped = defaultdict(list)
    for i in range(len(teachers)):
        key = (teachers[i][0], teachers[i][1])
        grouped[key].append(teachers[i][2])

    return render_template('teachers.html', title='Преподаватели', teachers=grouped)
```

editTeacher.py

```
from app import app
from flask import render_template
from flask_login import login_required

@app.get('/edit/teacher')
@login_required
def editTeacher():
    return render_template('editTeacher.html', title='Управление преподавателями')
```

updateTeacherForm.py

```
from app import app
from app.forms import UpdateTeacherForm
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/teacher/update', methods=['GET', 'POST'])
@login_required
def updateTeacherForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
password=app.config['DB_PASSWORD'],
                        dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        teachers = tupleProc(cur.execute('SELECT full_name FROM teacher ORDER BY full_name').fetchall())
        deps = tupleProc(cur.execute('SELECT dep_number FROM department ORDER BY dep_number').fetchall())

    if teachers is None or deps is None:
        abort(404)

    form = UpdateTeacherForm()
    form.name.choices = teachers
    form.dep.choices = deps

    if form.validate_on_submit():
        if form.name.data == 0:
            flash('Выберите преподавателя', 'danger')
            redirect(url_for('updateTeacherForm'))

        else:
            name = dict(form.name.choices).get(int(form.name.data))
            new_name = form.new_name.data or None
            pos = form.position.data or None
            ch = int(form.dep_ch.data)
            dep = dict(form.dep.choices).get(int(form.dep.data)) or None
            print(new_name, pos, ch, dep)
```

```

with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                    password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
    cur = con.cursor()

    # если задано новое имя
    if new_name is not None:
        cur.execute('UPDATE teacher '
                    'SET full_name = %s '
                    'WHERE full_name = %s', (new_name, name,))
        name = new_name

    # если задана должность
    if pos is not None:
        cur.execute('UPDATE teacher '
                    'SET position = %s '
                    'WHERE full_name = %s', (pos, name,))

    # если выбрана кафедра
    if dep is not None:
        # если надо изменить кафедру
        if ch == 1:
            cur.execute('UPDATE teacher_department '
                        'SET teacher_dep_number = %s '
                        'WHERE teacher_emp_record_num = '
                        '('
                        'SELECT emp_record_num '
                        'FROM teacher '
                        'WHERE full_name = %s'
                        ')', (dep, name,))

        # если надо добавить кафедру
        elif ch == 2:
            rec = cur.execute('SELECT emp_record_num '
                            'FROM teacher '
                            'WHERE full_name = %s', (name,)).fetchone()
            cur.execute('INSERT INTO teacher_department '
                        'VALUES (%s, %s)', (dep, rec[0],))

```

```

# если надо удалить кафедру
elif ch == 3:
    cur.execute('DELETE FROM teacher_department '
                'WHERE teacher_dep_number = %s AND teacher_emp_record_num = '
                '('
                'SELECT emp_record_num '
                'FROM teacher '
                'WHERE full_name = %s'
                ')', (dep, name,))

else:
    flash('Выберите действие для изменения кафедры', 'danger')
    redirect(url_for('updateTeacherForm'))

con.commit()

flash('Данные успешно обновлены', 'success')
redirect(url_for('updateTeacherForm'))

return render_template('renderForm.html', title='Редактирование существующего преподавателя', form=form)

```

insertTeacherForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg
from app.forms import InsertTeacherForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/teacher/insert', methods=['GET', 'POST'])
@login_required
def insertTeacherForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'],
                        dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        deps = tupleProc(cur.execute('SELECT dep_number FROM department ORDER BY dep_number').fetchall())

```

```

teachers = cur.execute('SELECT full_name, emp_record_num FROM teacher').fetchall()

if deps is None:
    abort(404)

form = InsertTeacherForm()
form.dep.choices = deps

if form.validate_on_submit():
    name = form.name.data
    rec = form.rec_num.data
    pos = form.position.data or None
    dep = dict(form.dep.choices).get(int(form.dep.data))
    t = (name, rec,)

    if teachers is not None and t in teachers:
        flash('Преподаватель с таким ФИО и номером трудовой книжки уже существует', 'danger')
        redirect(url_for('insertTeacherForm'))

    else:
        with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                             password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
            cur = con.cursor()
            cur.execute('INSERT INTO teacher (emp_record_num, full_name) '
                        'VALUES (%s, %s)', (rec, name,))

            # если указана должность
            if pos is not None:
                cur.execute('UPDATE teacher '
                            'SET position = %s '
                            'WHERE emp_record_num = %s', (pos, rec,))

            # если указана кафедра
            if dep != 'не выбрано':
                cur.execute('INSERT INTO teacher_department '
                            'VALUES (%s, %s)', (dep, rec))

```

```

flash('Данные успешно обновлены', 'success')
redirect(url_for('insertTeacherForm'))

return render_template('renderForm.html', title='Добавление нового преподавателя', form=form)

```

deleteTeacherForm.py

```

from app import app
from flask import render_template, flash, redirect, url_for, abort
import psycopg

from app.forms import DeleteTeacherForm
from app.tupleproc import tupleProc
from flask_login import login_required

@app.route('/edit/teacher/delete', methods=['GET', 'POST'])
@login_required
def deleteTeacherForm():
    with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                        password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
        cur = con.cursor()
        teachers = tupleProc(cur.execute('SELECT full_name FROM teacher ORDER BY full_name').fetchall())

    if teachers is None:
        abort(404)

    form = DeleteTeacherForm()
    form.name.choices = teachers

    if form.validate_on_submit():
        if form.name.data == 0:
            flash('Выберите преподавателя', 'danger')
            redirect(url_for('deleteTeacherForm'))
        else:
            tname = dict(form.name.choices).get(int(form.name.data))
            with psycopg.connect(host=app.config['DB_SERVER'], user=app.config['DB_USER'],
                                password=app.config['DB_PASSWORD'], dbname=app.config['DB_NAME']) as con:
                cur = con.cursor()
                cur.execute('DELETE FROM teacher '

```

```
'WHERE full_name = %s', (tname,))
```

```
flash('Данные успешно обновлены', 'success')
```

```
redirect(url_for('deleteTeacherForm'))
```

```
return render_template('renderForm.html', title='Удаление существующего преподавателя', form=form)
```

8 Описание сценариев использования

8.1 Пользователь без авторизации

Пользователь без авторизации может смотреть расписание групп, преподавателей и аудиторий, направления и их учебные планы и список преподавателей с указанием их кафедры и должности.

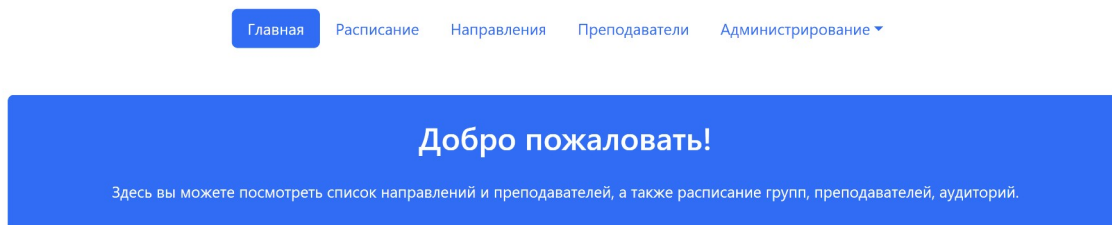


Рисунок 4 — Главная страница

The screenshot shows a form titled 'Параметры расписания'. At the top, there is a navigation bar with five items: 'Главная' (highlighted in blue), 'Расписание', 'Направления', 'Преподаватели', and 'Администрирование' (with a dropdown arrow). Below the navigation bar is the title 'Параметры расписания'. The form contains three dropdown menus: 'Группа' (with 'не выбрано' selected), 'Преподаватель' (with 'не выбрано' selected), and 'Корпус и аудитория' (with 'не выбрано' selected). Below these dropdowns is a blue button labeled 'Показать'.

Рисунок 5 — Форма выбора параметров расписания

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Расписание для группы 11

Суббота | Нечетная неделя

3 пара

Математический анализ

Васильев Василий Васильевич

Невский пр., 39 , 13-02

Вторник | Четная неделя

6 пара

Дискретная математика

Александров Александр Александрович

Московский пр., 3 , 112

Рисунок 6 — Просмотр расписания

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Направления

Код	Наименование	Учебный план
09.03.01	Информатика и вычислительная техника	2023 2024
09.03.02	Информационные системы и технологии	2024
09.03.04	Программная инженерия	2023
10.03.01	Информационная безопасность	2022
12.03.01	Приборостроение	2022
16.03.01	Техническая физика	2023 2024
27.03.03	Системный анализ и управление	2023

Рисунок 7 — Список направлений с ссылками на учебные планы

Учебный план направления 09.03.01 на 2024 год

№ семестра	Предмет	Тип аттестации	Лекционные часы	Лабораторные часы
1	Информатика	Экзамен	34	30
2	Информатика	Зачет	44	36

Рисунок 8 — Просмотр учебного плана

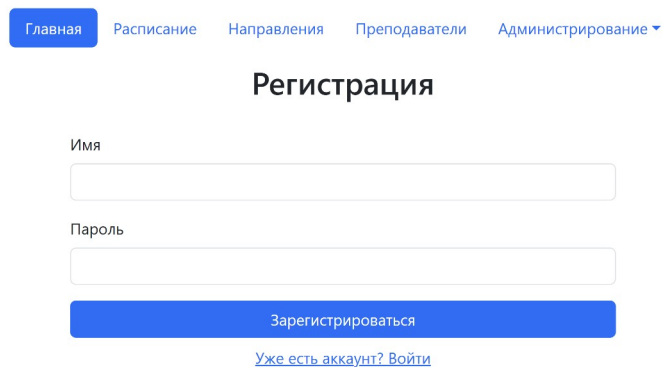
Преподаватели

ФИО	Должность	Кафедра
Александров Александр Александрович	Преподаватель	1
Андреев Андрей Андреевич	Профессор	4
Васильев Василий Васильевич	Доцент	2
Иванов Иван Иванович	Преподаватель	3
Иванова Мария Ивановна	Доцент	3 4
Петров Петр Петрович	Старший преподаватель	2
Сидорова Варвара Павловна	Старший преподаватель	2
Смирнова Екатерина Романовна	Профессор	1

Рисунок 9 — Список преподавателей

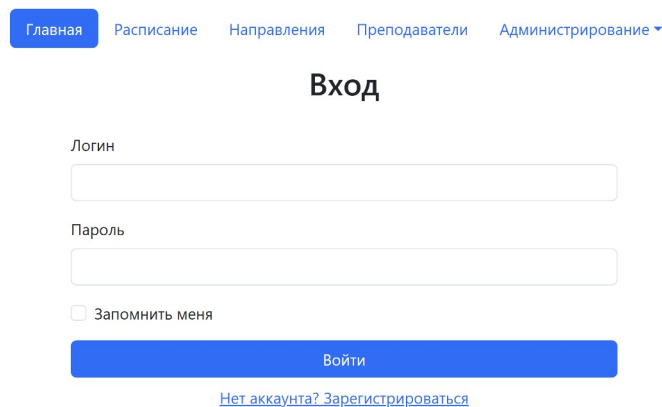
8.2 Администратор

После авторизации в системе администратор может просматривать список групп, а также изменять учебный план, расписание и информацию о преподавателях и группах.



The screenshot shows the registration form for an administrator. At the top, there is a navigation bar with five items: 'Главная' (highlighted in blue), 'Расписание', 'Направления', 'Преподаватели', and 'Администрирование' (with a dropdown arrow). Below the navigation bar is the title 'Регистрация'. The form consists of two input fields: 'Имя' (Name) and 'Пароль' (Password). Below these fields is a blue button labeled 'Зарегистрироваться'. At the bottom of the form, there is a link: 'Уже есть аккаунт? Войти'.

Рисунок 10 — Форма регистрации администратора



The screenshot shows the login form for an administrator. At the top, there is a navigation bar with five items: 'Главная' (highlighted in blue), 'Расписание', 'Направления', 'Преподаватели', and 'Администрирование' (with a dropdown arrow). Below the navigation bar is the title 'Вход'. The form consists of two input fields: 'Логин' (Login) and 'Пароль' (Password). Below these fields is a checkbox labeled 'Запомнить меня'. At the bottom of the form is a blue button labeled 'Войти'. Below the button, there is a link: 'Нет аккаунта? Зарегистрироваться'.

Рисунок 11 — Форма входа администратора

Добро пожаловать!

Здесь вы можете посмотреть список направлений и преподавателей, а также расписание групп, преподавателей, аудиторий.

Управление потоками, группами, преподавателями, предметами, расписанием и учебным планом находятся во вкладке "Администрирование".

Рисунок 12 — Главная страница администратора

Администрирование ▾

Управление потоками

Управление группами

Управление преподавателями

Управление расписанием

Управление предметами

Управление учебным планом

Выход

Рисунок 13 — Панель администратора

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Управление потоками

Добавить новый поток

Добавить

Удалить существующий поток

Удалить

Рисунок 14 — Управление потоками

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Добавление нового потока

Направление

не выбрано ▾

Год поступления

Добавить

Рисунок 15 — Добавление потока

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

При удалении потока будут удалены связанные с ним группы и их расписание!

Удаление существующего потока

Поток

не выбрано ▾

Удалить

Рисунок 16 — Удаление потока

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Управление группами

Посмотреть список групп

Список групп

Редактировать существующую группу

Редактировать

Добавить новую группу

Добавить

Удалить существующую группу

Удалить

Рисунок 17 — Управление группами

Главная

Расписание

Направления

Преподаватели

Администрирование

Группы

Номер группы	Текущий семестр	Код направления	Год поступления	Кафедра
10	5	12.03.01	2022	1
11	5	10.03.01	2022	3
12	3	27.03.03	2023	1
13	3	27.03.03	2023	1
20	1	16.03.01	2024	2
21	1	16.03.01	2024	2
22	3	16.03.01	2023	2
40	3	09.03.01	2023	4
41	1	09.03.01	2024	4
42	1	09.03.02	2024	4
44	3	09.03.04	2023	4

Рисунок 18 — Список групп

Главная

Расписание

Направления

Преподаватели

Администрирование

Редактирование существующей группы

Группа

не выбрано

Изменить текущий семестр на

Сохранить

Рисунок 19 — Редактирование группы

Нельзя добавить группу в несуществующий поток! [Добавить новый поток](#)

Добавление новой группы

Номер группы

Текущий семестр

Направление

Год поступления

Добавить

Рисунок 20 — Добавление группы

При удалении группы будет удалено связанное с ней расписание!

Удаление существующей группы

Группа

Удалить

Рисунок 21 — Удаление группы

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Управление преподавателями

Посмотреть список преподавателей

Список преподавателей

Редактировать существующего преподавателя

Редактировать

Добавить нового преподавателя

Добавить

Удалить существующего преподавателя

Удалить

Рисунок 22 — Управление преподавателями

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Редактирование существующего преподавателя

Преподаватель

не выбрано ▾

Новые ФИО

Изменить должность на

Изменить кафедру

не выбрано ▾

Номер кафедры

не выбрано ▾

Сохранить

Рисунок 23 — Редактирование преподавателя

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Добавление нового преподавателя

ФИО

Должность

Номер трудовой книжки

Номер кафедры

не выбрано ▾

Добавить

Рисунок 24 — Добавление преподавателя

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Удаление существующего преподавателя

Преподаватель

не выбрано ▾

Удалить

Рисунок 25 — Удаление преподавателя

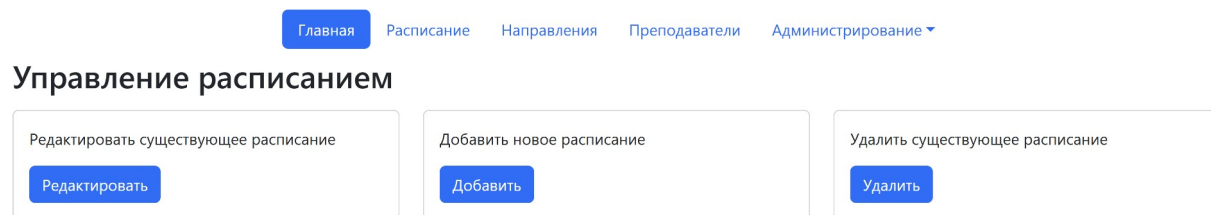


Рисунок 26 — Управление расписанием

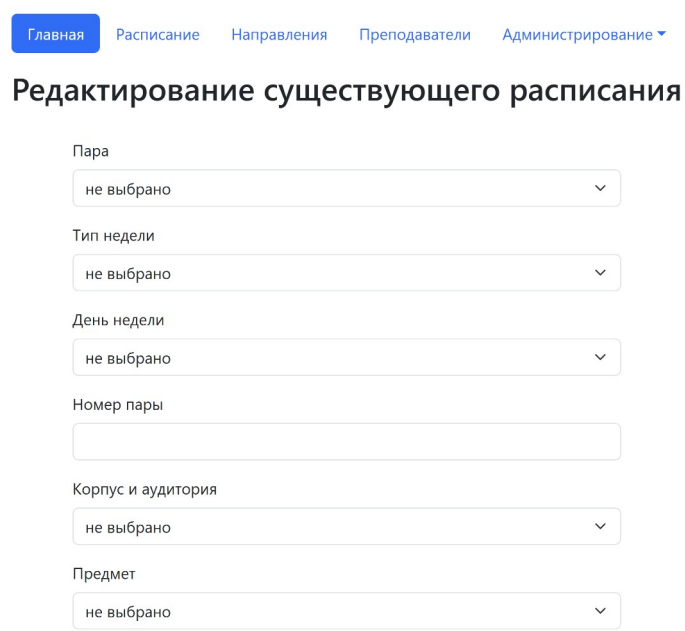


Рисунок 27 — Редактирование расписания

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Добавление нового расписания

Группа

не выбрано ▾

Тип недели

не выбрано ▾

День недели

не выбрано ▾

Номер пары

Корпус и аудитория

не выбрано ▾

Предмет

не выбрано ▾

Рисунок 28 — Добавление расписания

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Удаление существующего расписания

Группа

не выбрано ▾

Удалить

Рисунок 29 — Удаление расписания

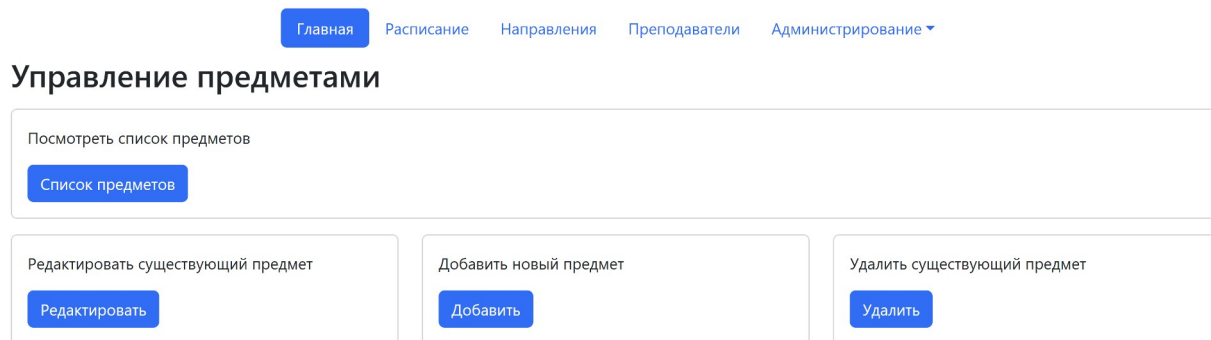


Рисунок 30 — Управление предметами

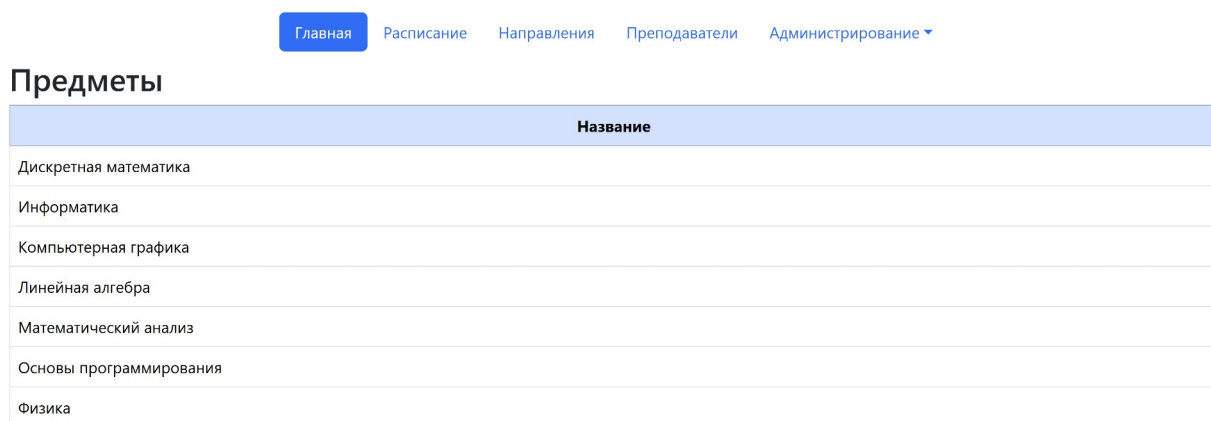


Рисунок 31 — Список предметов

[Главная](#) [Расписание](#) [Направления](#) [Преподаватели](#) [Администрирование ▾](#)

Редактирование существующего предмета

Название предмета

не выбрано ▾

Новое название

Сохранить

Рисунок 32 — Редактирование предмета

[Главная](#) [Расписание](#) [Направления](#) [Преподаватели](#) [Администрирование ▾](#)

Добавление нового предмета

Название предмета

Добавить

Рисунок 33 — Добавление предмета

При удалении предмета будут удалены связанные с ним пары и учебные планы!

Удаление существующего предмета

Название предмета

не выбрано ▾

Удалить

Рисунок 34 — Удаление предмета

Управление учебным планом

Посмотреть список направлений и их учебные планы

Учебные планы

Редактировать существующий учебный план

Редактировать

Добавить новый учебный план

Добавить

Удалить существующий учебный план

Удалить

Рисунок 35 — Управление учебным планом

Редактирование существующего учебного плана

Предмет

не выбрано ▾

Семестр

Количество лекционных часов

Количество лабораторных часов

Тип аттестации

не выбрано ▾

Сохранить

Рисунок 36 — Редактирование учебного плана

Добавление нового учебного плана

Поток

не выбрано ▾

Номер семестра

Предмет

не выбрано ▾

Тип аттестации

не выбрано ▾

Лекционные часы

Лабораторные часы

Добавить

Рисунок 37 — Добавление учебного плана

Главная

Расписание

Направления

Преподаватели

Администрирование ▾

Удаление существующего учебного плана

Предмет

не выбрано ▾

Удалить

Рисунок 38 — Удаление учебного плана

9 Вывод

В ходе выполнения курсовой работы была спроектирована база данных и разработано веб-приложение, которое позволяет пользователю просматривать расписание и учебные планы университета. Приложение и база данных развернуты на бесплатных сервисах, успешно работают и соответствуют требованиям технического задания на курсовую работу.